DOCUMENTACIÓN TÉCNICA DEL PROYECTO · UADY

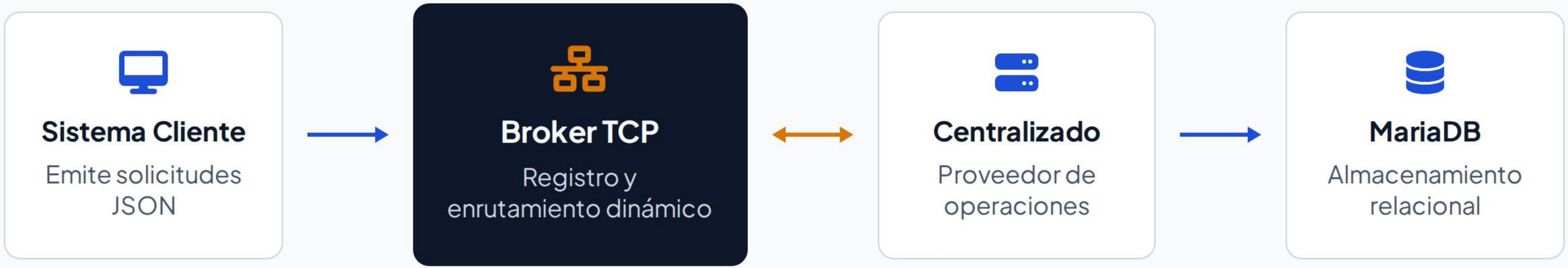
# Sistema de Inventario y Almacenes

Arquitectura distribuida sobre TCP/JSON con Broker central, servicios desacoplados y persistencia en MariaDB.

`broker · common · sistema-centralizado · sistema-cliente`

VISIÓN ESTRUCTURAL

# Arquitectura del Sistema




Desacoplamiento total entre consumidores y proveedores mediante sockets TCP persistentes.

ORGANIZACIÓN DEL CÓDIGO

# Módulos del Repositorio

### `broker/`

Servidor TCP y enrutador central.

### `common/`

Contratos JSON y modelos compartidos.

### `docker/`

Esquema SQL y carga inicial.

### `docs/`

Diagramas Mermaid y pruebas ejecutadas.

MÓDULO COMMON · CONTRATO

# Protocolo Solicitud y Respuesta

**Mensajería JSON:** Intercambio estructurado sobre sockets TCP.

**GsonFactory:** Serialización uniforme con soporte `LocalDateTime`.

**Control de Estado:** Tipado mediante `EstadoRespuesta` y `CodigoError`.

```
mx.uady.inventario.common.contrato

public class Solicitud {
    private String servicio;
    private String operacion;
    private Map parametros;
}

public class Respuesta {
    private EstadoRespuesta estado;
    private Object datos;
    private CodigoError error;
}
```

MÓDULO BROKER · ROUTING

# Enrutamiento Dinámico de Servicios

mx.uady.inventario.broker.routing.Enrutador

```
public Respuesta enrutar(Solicitud req) {
    Proveedor destino = registro
        .obtenerProveedor(req.getServicio());
    if (destino == null) {
        return Respuesta.error(
            CodigoError.SERVICIO_DESCONOCIDO);
    }
    return destino.enviar(req);
}
```

**RegistroServicios:** Tabla concurrente de proveedores activos.

**ManejadorConexion:** Atención paralela por hilo de cliente.

**Cambio en Caliente:** Reemplazo y remoción sin detener tráfico.

BASE DE DATOS RELACIONAL

# Persistencia en MariaDB

**Catálogo:** `Producto` e `HistorialPrecio`.

**Existencias:** `Mercancia` vinculada a `TiendaAlmacen`.

**Operación:** Registro de `Usuario`, `Rol` y `Venta`.

docker/mariadb/init/01_schema.sql

```
CREATE TABLE mercancia (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto_id INT NOT NULL,
    tienda_id INT NOT NULL,
    estado VARCHAR(32) NOT NULL,
    codigo_qr VARCHAR(128) UNIQUE,
    FOREIGN KEY (producto_id)
        REFERENCES producto(id)
);
```

DIAGRAMAS DE SECUENCIA

# Flujos de Operación del Sistema

01

### Topología Broker

Registrar, actualizar y remover proveedores.

02

### Consultas Base

Listar tiendas, usuarios e inventario.

03

### Movimientos

Actualizar tienda de venta y precios.

04

### Identificación QR

Obtener código QR de cada mercancía.

INFRAESTRUCTURA COMO CÓDIGO

# Despliegue con Docker Compose

docker-compose.yml

```
services:
  mariadb:
    image: mariadb:11
    volumes:
      - ./docker/mariadb/init:/docker-entrypoint-initdb.d
  broker:
    build: ./broker
    ports:
      - "${BROKER_PORT}:9000"
```

**Arranque Automático:** Carga `01_schema.sql` y `02_seed.sql`.

**Variables Seguras:** Parámetros centralizados en `.env.example`.

**Portabilidad:** Construcción reproducible con Maven y Docker.

ASEGURAMIENTO DE CALIDAD

# Cobertura de Documentación y Pruebas

**14**

Evidencias de ejecución

**9**

Secuencias modeladas

**8**

Suites JUnit automatizadas

EVIDENCIAS DE PRUEBA

# Tolerancia a Fallos y Concurrencia

## Resiliencia ante Fallos

**Servicio Desconocido:** Respuesta controlada sin interrupciones.

**Proveedor Caído:** Manejo de timeout y conmutación dinámica.

**Solicitud Duplicada:** Validación idempotente ante reintentos.

## Concurrencia y Carga

**Hilos Simultáneos:** Validado con `EvidenciaConcurrencia.java`.

**Latencia Medida:** Benchmark en `EvidenciaRendimiento.java`.

**Reproducibilidad:** Scripts automatizados `parte1.sh` – `parte3.sh`.

INVENTARIO Y ALMACENES · UADY

# Arquitectura Modular y Resiliente

Documentación técnica, contratos de código y evidencias de prueba consolidados.